INTRODUCTION

The Travel Booking System is a database-driven application designed to manage and streamline travel-related services such as flight bookings, hotel reservations, customer details, payments, and ticket management. The system is implemented using Structured Query Language (SQL) to efficiently store, retrieve, update, and manage large amounts of travel data in a structured format.

In today's digital era, online travel booking platforms have replaced traditional manual reservation systems. Managing such systems requires a well-designed database to handle multiple entities like customers, bookings, destinations, payments, and schedules while maintaining data integrity and minimizing redundancy.

This project demonstrates the practical implementation of Database Management System (DBMS) concepts such as:

Entity-Relationship (ER) modelling

Normalization

Primary and Foreign Keys

Constraints

SQL Queries (SELECT, INSERT, UPDATE, DELETE)

Joins and Aggregate Functions

The Travel Booking System ensures accurate record maintenance, faster query processing, secure data handling, and efficient transaction management. The main objective of this project is to design a relational database schema and perform various SQL operations to simulate real-world travel booking functionalities.

Table Structure Explanation

The Travel Booking System database is designed using a relational model consisting of four main tables: Customers, Destinations, Bookings, and Payments. These tables are connected using primary keys and foreign keys to maintain referential integrity and ensure structured data management.

1. Customers Table

The Customers table stores all information related to users who book travel services.
Attributes:

- Customer ID – Primary Key; uniquely identifies each customer.
- Name – Stores the full name of the customer (NOT NULL constraint applied).
- Email – Stores the email address (UNIQUE constraint ensures no duplicate emails).
- Phone – Stores contact number.
- City – Stores the customer's city of residence.

This table ensures that each customer is uniquely identified and prevents duplicate email registrations using the UNIQUE constraint.

2. Destinations Table

The Destinations table stores details of available travel destinations. Attributes:

- DestinationID – Primary Key; uniquely identifies each destination.
- CityName – Name of the destination city (NOT NULL).
- Country – Country where the destination is located.
- PricePerDay – Cost per day for staying at the destination (DECIMAL datatype used for accuracy in monetary values).

This table allows efficient storage of destination pricing and location data.

---

3. Bookings Table

The Bookings table records travel bookings made by customers. It acts as a bridge between Customers and Destinations. Attributes:

- BookingID – Primary Key; uniquely identifies each booking.

- CustomerID – Foreign Key referencing Customers(CustomerID).
- DestinationID – Foreign Key referencing Destinations(DestinationID).
- TravelDate – Date of travel.
- DurationDays – Number of days booked. Foreign key constraints ensure:
- A booking cannot exist without a valid customer.
- A booking cannot exist without a valid destination.

This maintains referential integrity and prevents invalid data entry.

4. Payments Table

The Payments table stores payment details for each booking.

Attributes:

- PaymentID – Primary Key; uniquely identifies each payment.
- BookingID – Foreign Key referencing Bookings(BookingID).
- Amount – Total payment amount (DECIMAL used for precision).
- PaymentMethod – Mode of payment (e.g., UPI, Credit Card, Debit Card, Bank Transfer).

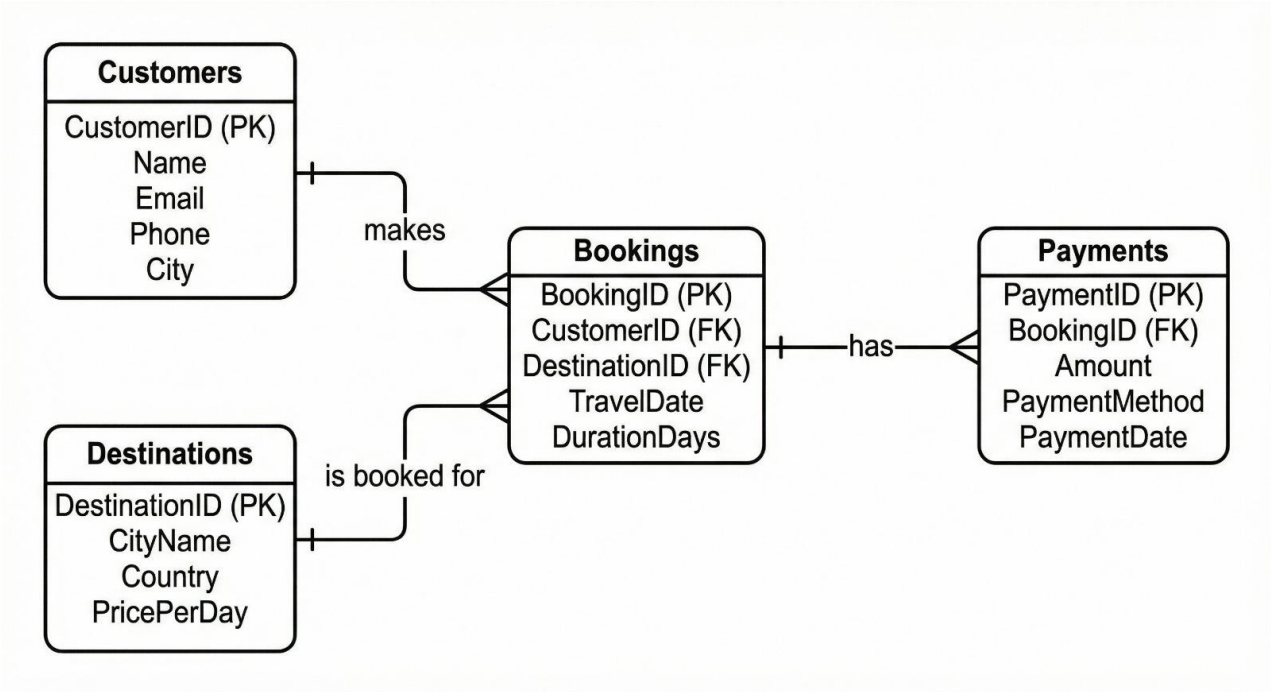
- PaymentDate – Date when payment was made.

This table ensures that every payment is linked to a valid booking.

Relationships Between Tables

- One Customer can make multiple Bookings (One-toMany relationship).
- One Destination can be booked multiple times (One-toMany relationship).
- One Booking has one associated Payment (One-to-One relationship in this design).

The database design follows normalization principles to reduce redundancy and improve consistency. Primary and
foreign key constraints ensure data integrity, while appropriate data types enhance performance and accuracy.



1. View all customer details:

```
MariaDB [(none)]> USE TravelBookingSystem;
Database changed
MariaDB [TravelBookingSystem]> SELECT * FROM Customers;
+------------+----------------+--------------------+------------+-----------+
| CustomerID | Name           | Email              | Phone      | City      |
+------------+----------------+--------------------+------------+-----------+
|          1 | Amit Sharma    | amit@example.com   | 9876543210 | Delhi     |
|          2 | Priya Singh    | priya@example.com  | 9876543211 | Mumbai    |
|          3 | Rahul Verma    | rahul@example.com  | 9876543212 | Kolkata   |
|          4 | Sneha Gupta    | sneha@example.com  | 9876543213 | Chennai   |
|          5 | Vikram Patel   | vikram@example.com | 9876543214 | Ahmedabad |
|          6 | Pooja Reddy    | pooja@example.com  | 9876543215 | Hyderabad |
|          7 | Karan Malhotra | karan@example.com  | 9876543216 | Pune      |
|          8 | Anjali Desai   | anjali@example.com | 9876543217 | Surat     |
|          9 | Rohan Joshi    | rohan@example.com  | 9876543218 | Jaipur    |
|         10 | Neha Kapoor    | neha@example.com   | 9876543219 | Lucknow   |
+------------+----------------+--------------------+------------+-----------+
10 rows in set (0.089 sec)
```

2. View all destination offerings:

```
MariaDB [TravelBookingSystem]> SELECT * FROM Destinations;
+---------------+----------+-----------+-------------+
| DestinationID | CityName | Country   | PricePerDay |
+---------------+----------+-----------+-------------+
|           101 | Paris    | France    |      150.00 |
|           102 | Tokyo    | Japan     |      200.00 |
|           103 | New York | USA       |      250.00 |
|           104 | Rome     | Italy     |      180.00 |
|           105 | Sydney   | Australia |      220.00 |
|           106 | Dubai    | UAE       |      300.00 |
|           107 | Bali     | Indonesia |      120.00 |
|           108 | Bangkok  | Thailand  |      100.00 |
|           109 | London   | UK        |      210.00 |
|           110 | Malé     | Maldives  |      350.00 |
+---------------+----------+-----------+-------------+
10 rows in set (0.003 sec)
```

3. List all booking records:

```
MariaDB [TravelBookingSystem]> SELECT * FROM Bookings;
+-----------+------------+---------------+------------+--------------+
| BookingID | CustomerID | DestinationID | TravelDate | DurationDays |
+-----------+------------+---------------+------------+--------------+
|      1001 |          1 |           101 | 2026-03-15 |            5 |
|      1002 |          2 |           107 | 2026-04-10 |            7 |
|      1003 |          3 |           105 | 2026-05-20 |           10 |
|      1004 |          4 |           102 | 2026-06-05 |            8 |
|      1005 |          5 |           106 | 2026-07-12 |            4 |
|      1006 |          6 |           110 | 2026-08-18 |            6 |
|      1007 |          7 |           103 | 2026-09-22 |            7 |
|      1008 |          8 |           104 | 2026-10-30 |            5 |
|      1009 |          9 |           108 | 2026-11-14 |            9 |
|      1010 |         10 |           109 | 2026-12-25 |            6 |
+-----------+------------+---------------+------------+--------------+
10 rows in set (0.005 sec)
```

4. Review all payment transactions:

```
MariaDB [TravelBookingSystem]> SELECT * FROM Payments;
+-----------+-----------+---------+---------------+-------------+
| PaymentID | BookingID | Amount  | PaymentMethod | PaymentDate |
+-----------+-----------+---------+---------------+-------------+
|      5001 |      1001 |  750.00 | Credit Card   | 2026-02-26  |
|      5002 |      1002 |  840.00 | UPI           | 2026-02-26  |
|      5003 |      1003 | 2200.00 | Bank Transfer | 2026-02-26  |
|      5004 |      1004 | 1600.00 | Credit Card   | 2026-02-26  |
|      5005 |      1005 | 1200.00 | Debit Card    | 2026-02-26  |
|      5006 |      1006 | 2100.00 | UPI           | 2026-02-26  |
|      5007 |      1007 | 1750.00 | Credit Card   | 2026-02-26  |
|      5008 |      1008 |  900.00 | Debit Card    | 2026-02-26  |
|      5009 |      1009 |  900.00 | UPI           | 2026-02-26  |
|      5010 |      1010 | 1260.00 | Credit Card   | 2026-02-26  |
+-----------+-----------+---------+---------------+-------------+
10 rows in set (0.003 sec)
```

5. Fetch only the names and emails of customers:

```
MariaDB [TravelBookingSystem]> SELECT Name, Email FROM Customers;
+----------------+--------------------+
| Name           | Email              |
+----------------+--------------------+
| Amit Sharma    | amit@example.com   |
| Priya Singh    | priya@example.com  |
| Rahul Verma    | rahul@example.com  |
| Sneha Gupta    | sneha@example.com  |
| Vikram Patel   | vikram@example.com |
| Pooja Reddy    | pooja@example.com  |
| Karan Malhotra | karan@example.com  |
| Anjali Desai   | anjali@example.com |
| Rohan Joshi    | rohan@example.com  |
| Neha Kapoor    | neha@example.com   |
+----------------+--------------------+
10 rows in set (0.002 sec)
```

6. Show destination cities and their daily prices:

```
MariaDB [TravelBookingSystem]> SELECT CityName, PricePerDay FROM Destinations;
+----------+-------------+
| CityName | PricePerDay |
+----------+-------------+
| Paris    |      150.00 |
| Tokyo    |      200.00 |
| New York |      250.00 |
| Rome     |      180.00 |
| Sydney   |      220.00 |
| Dubai    |      300.00 |
| Bali     |      120.00 |
| Bangkok  |      100.00 |
| London   |      210.00 |
| Malé     |      350.00 |
+----------+-------------+
10 rows in set (0.001 sec)
```

7. Find all customers living in 'Mumbai':

```
MariaDB [TravelBookingSystem]> SELECT * FROM Customers WHERE City = 'Mumbai';
+------------+-------------+-------------------+------------+--------+
| CustomerID | Name        | Email             | Phone      | City   |
+------------+-------------+-------------------+------------+--------+
|          2 | Priya Singh | priya@example.com | 9876543211 | Mumbai |
+------------+-------------+-------------------+------------+--------+
1 row in set (0.003 sec)
```

8. List destinations that cost more than Rs.200 per day:

```
MariaDB [TravelBookingSystem]> SELECT * FROM Destinations WHERE PricePerDay > 200.00;
+---------------+----------+-----------+-------------+
| DestinationID | CityName | Country   | PricePerDay |
+---------------+----------+-----------+-------------+
|           103 | New York | USA       |      250.00 |
|           105 | Sydney   | Australia |      220.00 |
|           106 | Dubai    | UAE       |      300.00 |
|           109 | London   | UK        |      210.00 |
|           110 | Malé     | Maldives  |      350.00 |
+---------------+----------+-----------+-------------+
5 rows in set (0.001 sec)
```

9. Find bookings for a duration of exactly 7 days:

```
MariaDB [TravelBookingSystem]> SELECT * FROM Bookings WHERE DurationDays = 7;
+-----------+------------+---------------+------------+--------------+
| BookingID | CustomerID | DestinationID | TravelDate | DurationDays |
+-----------+------------+---------------+------------+--------------+
|      1002 |          2 |           107 | 2026-04-10 |            7 |
|      1007 |          7 |           103 | 2026-09-22 |            7 |
+-----------+------------+---------------+------------+--------------+
2 rows in set (0.001 sec)
```

10. Retrieve payments made via 'UPI':

```
MariaDB [TravelBookingSystem]> SELECT * FROM Payments WHERE PaymentMethod = 'UPI';
+-----------+-----------+---------+---------------+-------------+
| PaymentID | BookingID | Amount  | PaymentMethod | PaymentDate |
+-----------+-----------+---------+---------------+-------------+
|      5002 |      1002 |  840.00 | UPI           | 2026-02-26  |
|      5006 |      1006 | 2100.00 | UPI           | 2026-02-26  |
|      5009 |      1009 |  900.00 | UPI           | 2026-02-26  |
+-----------+-----------+---------+---------------+-------------+
3 rows in set (0.001 sec)
```

11. Find destinations located in 'France' or 'Italy':

```
MariaDB [TravelBookingSystem]> SELECT * FROM Destinations WHERE Country IN ('France', 'Italy');
+---------------+----------+---------+-------------+
| DestinationID | CityName | Country | PricePerDay |
+---------------+----------+---------+-------------+
|           101 | Paris    | France  |      150.00 |
|           104 | Rome     | Italy   |      180.00 |
+---------------+----------+---------+-------------+
2 rows in set (0.001 sec)
```

12. List bookings scheduled after June 1, 2026:

```
MariaDB [TravelBookingSystem]> SELECT * FROM Bookings WHERE TravelDate > '2026-06-01';
+-----------+------------+---------------+------------+--------------+
| BookingID | CustomerID | DestinationID | TravelDate | DurationDays |
+-----------+------------+---------------+------------+--------------+
|      1004 |          4 |           102 | 2026-06-05 |            8 |
|      1005 |          5 |           106 | 2026-07-12 |            4 |
|      1006 |          6 |           110 | 2026-08-18 |            6 |
|      1007 |          7 |           103 | 2026-09-22 |            7 |
|      1008 |          8 |           104 | 2026-10-30 |            5 |
|      1009 |          9 |           108 | 2026-11-14 |            9 |
|      1010 |         10 |           109 | 2026-12-25 |            6 |
+-----------+------------+---------------+------------+--------------+
7 rows in set (0.001 sec)
```

13. Find payments exceeding $1500:

```
MariaDB [TravelBookingSystem]> SELECT * FROM Payments WHERE Amount > 1500.00;
+-----------+-----------+---------+---------------+-------------+
| PaymentID | BookingID | Amount  | PaymentMethod | PaymentDate |
+-----------+-----------+---------+---------------+-------------+
|      5003 |      1003 | 2200.00 | Bank Transfer | 2026-02-26  |
|      5004 |      1004 | 1600.00 | Credit Card   | 2026-02-26  |
|      5006 |      1006 | 2100.00 | UPI           | 2026-02-26  |
|      5007 |      1007 | 1750.00 | Credit Card   | 2026-02-26  |
+-----------+-----------+---------+---------------+-------------+
4 rows in set (0.001 sec)
```

14. **Find customers whose names start with 'A':**

```
MariaDB [TravelBookingSystem]> SELECT * FROM Customers WHERE Name LIKE 'A%';
+------------+--------------+--------------------+------------+-------+
| CustomerID | Name         | Email              | Phone      | City  |
+------------+--------------+--------------------+------------+-------+
|          1 | Amit Sharma  | amit@example.com   | 9876543210 | Delhi |
|          8 | Anjali Desai | anjali@example.com | 9876543217 | Surat |
+------------+--------------+--------------------+------------+-------+
2 rows in set (0.001 sec)
```

15. List customers alphabetically by name:

```
MariaDB [TravelBookingSystem]> SELECT * FROM Customers ORDER BY Name ASC;
+------------+----------------+--------------------+------------+-----------+
| CustomerID | Name           | Email              | Phone      | City      |
+------------+----------------+--------------------+------------+-----------+
|          1 | Amit Sharma    | amit@example.com   | 9876543210 | Delhi     |
|          8 | Anjali Desai   | anjali@example.com | 9876543217 | Surat     |
|          7 | Karan Malhotra | karan@example.com  | 9876543216 | Pune      |
|         10 | Neha Kapoor    | neha@example.com   | 9876543219 | Lucknow   |
|          6 | Pooja Reddy    | pooja@example.com  | 9876543215 | Hyderabad |
|          2 | Priya Singh    | priya@example.com  | 9876543211 | Mumbai    |
|          3 | Rahul Verma    | rahul@example.com  | 9876543212 | Kolkata   |
|          9 | Rohan Joshi    | rohan@example.com  | 9876543218 | Jaipur    |
|          4 | Sneha Gupta    | sneha@example.com  | 9876543213 | Chennai   |
|          5 | Vikram Patel   | vikram@example.com | 9876543214 | Ahmedabad |
+------------+----------------+--------------------+------------+-----------+
10 rows in set (0.001 sec)
```

16. Display destinations from most expensive to least expensive:

```
MariaDB [TravelBookingSystem]> SELECT * FROM Destinations ORDER BY PricePerDay DESC;
+---------------+----------+-----------+-------------+
| DestinationID | CityName | Country   | PricePerDay |
+---------------+----------+-----------+-------------+
|           110 | Malé     | Maldives  |      350.00 |
|           106 | Dubai    | UAE       |      300.00 |
|           103 | New York | USA       |      250.00 |
|           105 | Sydney   | Australia |      220.00 |
|           109 | London   | UK        |      210.00 |
|           102 | Tokyo    | Japan     |      200.00 |
|           104 | Rome     | Italy     |      180.00 |
|           101 | Paris    | France    |      150.00 |
|           107 | Bali     | Indonesia |      120.00 |
|           108 | Bangkok  | Thailand  |      100.00 |
+---------------+----------+-----------+-------------+
10 rows in set (0.001 sec)
```

17. Show the 3 most recent payments:

```
MariaDB [TravelBookingSystem]> SELECT * FROM Payments ORDER BY PaymentDate DESC LIMIT 3;
+-----------+-----------+---------+---------------+-------------+
| PaymentID | BookingID | Amount  | PaymentMethod | PaymentDate |
+-----------+-----------+---------+---------------+-------------+
|      5010 |      1010 | 1260.00 | Credit Card   | 2026-02-26  |
|      5002 |      1002 |  840.00 | UPI           | 2026-02-26  |
|      5003 |      1003 | 2200.00 | Bank Transfer | 2026-02-26  |
+-----------+-----------+---------+---------------+-------------+
3 rows in set (0.001 sec)
```

18. List the top 5 longest trips booked:

```
MariaDB [TravelBookingSystem]> SELECT * FROM Bookings ORDER BY DurationDays DESC LIMIT 5;
+-----------+------------+---------------+------------+--------------+
| BookingID | CustomerID | DestinationID | TravelDate | DurationDays |
+-----------+------------+---------------+------------+--------------+
|      1003 |          3 |           105 | 2026-05-20 |           10 |
|      1009 |          9 |           108 | 2026-11-14 |            9 |
|      1004 |          4 |           102 | 2026-06-05 |            8 |
|      1002 |          2 |           107 | 2026-04-10 |            7 |
|      1007 |          7 |           103 | 2026-09-22 |            7 |
+-----------+------------+---------------+------------+--------------+
5 rows in set (0.001 sec)
```

19. Count the total number of registered customers:

```
MariaDB [TravelBookingSystem]> SELECT COUNT(CustomerID) AS TotalCustomers FROM Customers;
+----------------+
| TotalCustomers |
+----------------+
|             10 |
+----------------+
1 row in set (0.013 sec)
```

20. Calculate the average price per day across all destinations:

```
MariaDB [TravelBookingSystem]> SELECT AVG(PricePerDay) AS AveragePrice FROM Destinations;
+--------------+
| AveragePrice |
+--------------+
|   208.000000 |
+--------------+
1 row in set (0.001 sec)
```

21. Calculate the total revenue generated from all payments:

```
MariaDB [TravelBookingSystem]> SELECT SUM(Amount) AS TotalRevenue FROM Payments;
+--------------+
| TotalRevenue |
+--------------+
|     13500.00 |
+--------------+
1 row in set (0.001 sec)
```

22. Find the highest single payment amount:

```
MariaDB [TravelBookingSystem]> SELECT MAX(Amount) AS HighestPayment FROM Payments;
+----------------+
| HighestPayment |
+----------------+
|        2200.00 |
+----------------+
1 row in set (0.001 sec)
```

23. Find the shortest booking duration in days:

```
MariaDB [TravelBookingSystem]> SELECT MIN(DurationDays) AS ShortestTrip FROM Bookings;
+--------------+
| ShortestTrip |
+--------------+
|            4 |
+--------------+
1 row in set (0.001 sec)
```

24. Count how many customers are in each city:

```
MariaDB [TravelBookingSystem]> SELECT City, COUNT(CustomerID) AS NumberOfCustomers
    -> FROM Customers GROUP BY City;
+-----------+-------------------+
| City      | NumberOfCustomers |
+-----------+-------------------+
| Ahmedabad |                 1 |
| Chennai   |                 1 |
| Delhi     |                 1 |
| Hyderabad |                 1 |
| Jaipur    |                 1 |
| Kolkata   |                 1 |
| Lucknow   |                 1 |
| Mumbai    |                 1 |
| Pune      |                 1 |
| Surat     |                 1 |
+-----------+-------------------+
10 rows in set (0.001 sec)
```

25. Show the number of destinations available per country:

```
MariaDB [TravelBookingSystem]> SELECT Country, COUNT(DestinationID) AS NumDestinations
    -> FROM Destinations GROUP BY Country;
+-----------+-----------------+
| Country   | NumDestinations |
+-----------+-----------------+
| Australia |               1 |
| France    |               1 |
| Indonesia |               1 |
| Italy     |               1 |
| Japan     |               1 |
| Maldives  |               1 |
| Thailand  |               1 |
| UAE       |               1 |
| UK        |               1 |
| USA       |               1 |
+-----------+-----------------+
10 rows in set (0.001 sec)
```

26. Calculate total revenue generated per payment method:

```
MariaDB [TravelBookingSystem]> SELECT PaymentMethod, SUM(Amount) AS TotalRevenue
    -> FROM Payments GROUP BY PaymentMethod;
+---------------+--------------+
| PaymentMethod | TotalRevenue |
+---------------+--------------+
| Bank Transfer |      2200.00 |
| Credit Card   |      5360.00 |
| Debit Card    |      2100.00 |
| UPI           |      3840.00 |
+---------------+--------------+
4 rows in set (0.001 sec)
```

27. Find payment methods that have generated more than Rs.2000 total:

```
MariaDB [TravelBookingSystem]> SELECT PaymentMethod, SUM(Amount) AS TotalRevenue
    -> FROM Payments GROUP BY PaymentMethod HAVING SUM(Amount) > 2000;
+---------------+--------------+
| PaymentMethod | TotalRevenue |
+---------------+--------------+
| Bank Transfer |      2200.00 |
| Credit Card   |      5360.00 |
| Debit Card    |      2100.00 |
| UPI           |      3840.00 |
+---------------+--------------+
4 rows in set (0.017 sec)
```

28. Calculate the average booking duration per destination ID:

```
MariaDB [TravelBookingSystem]> SELECT DestinationID, AVG(DurationDays) AS AvgDuration
    -> FROM Bookings GROUP BY DestinationID;
+---------------+-------------+
| DestinationID | AvgDuration |
+---------------+-------------+
|           101 |      5.0000 |
|           102 |      8.0000 |
|           103 |      7.0000 |
|           104 |      5.0000 |
|           105 |     10.0000 |
|           106 |      4.0000 |
|           107 |      7.0000 |
|           108 |      9.0000 |
|           109 |      6.0000 |
|           110 |      6.0000 |
+---------------+-------------+
10 rows in set (0.006 sec)
```

29. Show booking IDs alongside the corresponding customer names:

```
MariaDB [TravelBookingSystem]> SELECT Bookings.BookingID, Customers.Name
    -> FROM Bookings
    -> JOIN Customers ON Bookings.CustomerID = Customers.CustomerID;
+-----------+----------------+
| BookingID | Name           |
+-----------+----------------+
|      1001 | Amit Sharma    |
|      1002 | Priya Singh    |
|      1003 | Rahul Verma    |
|      1004 | Sneha Gupta    |
|      1005 | Vikram Patel   |
|      1006 | Pooja Reddy    |
|      1007 | Karan Malhotra |
|      1008 | Anjali Desai   |
|      1009 | Rohan Joshi    |
|      1010 | Neha Kapoor    |
+-----------+----------------+
10 rows in set (0.002 sec)
```

30. Display travel dates and the destination city names:

```
MariaDB [TravelBookingSystem]> SELECT Bookings.TravelDate, Destinations.CityName
    -> FROM Bookings
    -> JOIN Destinations ON Bookings.DestinationID = Destinations.DestinationID;
+------------+----------+
| TravelDate | CityName |
+------------+----------+
| 2026-03-15 | Paris    |
| 2026-04-10 | Bali     |
| 2026-05-20 | Sydney   |
| 2026-06-05 | Tokyo    |
| 2026-07-12 | Dubai    |
| 2026-08-18 | Malé     |
| 2026-09-22 | New York |
| 2026-10-30 | Rome     |
| 2026-11-14 | Bangkok  |
| 2026-12-25 | London   |
+------------+----------+
10 rows in set (0.001 sec)
```

31. List all payment details along with the travel date they paid for:

```
MariaDB [TravelBookingSystem]> SELECT Payments.PaymentID, Payments.Amount, Bookings.TravelDate
    -> FROM Payments
    -> JOIN Bookings ON Payments.BookingID = Bookings.BookingID;
+-----------+---------+------------+
| PaymentID | Amount  | TravelDate |
+-----------+---------+------------+
|      5001 |  750.00 | 2026-03-15 |
|      5002 |  840.00 | 2026-04-10 |
|      5003 | 2200.00 | 2026-05-20 |
|      5004 | 1600.00 | 2026-06-05 |
|      5005 | 1200.00 | 2026-07-12 |
|      5006 | 2100.00 | 2026-08-18 |
|      5007 | 1750.00 | 2026-09-22 |
|      5008 |  900.00 | 2026-10-30 |
|      5009 |  900.00 | 2026-11-14 |
|      5010 | 1260.00 | 2026-12-25 |
+-----------+---------+------------+
10 rows in set (0.001 sec)
```

32. Complete Itinerary (Join 3 tables): Customer Name, Destination City, and Travel Date:

```
MariaDB [TravelBookingSystem]> SELECT Customers.Name, Destinations.CityName, Bookings.TravelDate
    -> FROM Bookings
    -> JOIN Customers ON Bookings.CustomerID = Customers.CustomerID
    -> JOIN Destinations ON Bookings.DestinationID = Destinations.DestinationID;
+----------------+----------+------------+
| Name           | CityName | TravelDate |
+----------------+----------+------------+
| Amit Sharma    | Paris    | 2026-03-15 |
| Priya Singh    | Bali     | 2026-04-10 |
| Rahul Verma    | Sydney   | 2026-05-20 |
| Sneha Gupta    | Tokyo    | 2026-06-05 |
| Vikram Patel   | Dubai    | 2026-07-12 |
| Pooja Reddy    | Malé     | 2026-08-18 |
| Karan Malhotra | New York | 2026-09-22 |
| Anjali Desai   | Rome     | 2026-10-30 |
| Rohan Joshi    | Bangkok  | 2026-11-14 |
| Neha Kapoor    | London   | 2026-12-25 |
+----------------+----------+------------+
10 rows in set (0.001 sec)
```

33. Financial Report (Join 3 tables): Customer Name, Payment Method, and Amount:

```
MariaDB [TravelBookingSystem]> SELECT Customers.Name, Payments.PaymentMethod, Payments.Amount
    -> FROM Payments
    -> JOIN Bookings ON Payments.BookingID = Bookings.BookingID
    -> JOIN Customers ON Bookings.CustomerID = Customers.CustomerID;
+----------------+---------------+---------+
| Name           | PaymentMethod | Amount  |
+----------------+---------------+---------+
| Amit Sharma    | Credit Card   |  750.00 |
| Priya Singh    | UPI           |  840.00 |
| Rahul Verma    | Bank Transfer | 2200.00 |
| Sneha Gupta    | Credit Card   | 1600.00 |
| Vikram Patel   | Debit Card    | 1200.00 |
| Pooja Reddy    | UPI           | 2100.00 |
| Karan Malhotra | Credit Card   | 1750.00 |
| Anjali Desai   | Debit Card    |  900.00 |
| Rohan Joshi    | UPI           |  900.00 |
| Neha Kapoor    | Credit Card   | 1260.00 |
+----------------+---------------+---------+
10 rows in set (0.002 sec)
```

34. Find out which customer booked the trip to 'Paris':

```
MariaDB [TravelBookingSystem]> SELECT Customers.Name
    -> FROM Customers
    -> JOIN Bookings ON Customers.CustomerID = Bookings.CustomerID
    -> JOIN Destinations ON Bookings.DestinationID = Destinations.DestinationID
    -> WHERE Destinations.CityName = 'Paris';
+-------------+
| Name        |
+-------------+
| Amit Sharma |
+-------------+
1 row in set (0.002 sec)
```

35. Find customers who booked a trip longer than the average trip duration:

```
MariaDB [TravelBookingSystem]> SELECT Name FROM Customers WHERE CustomerID IN (
    ->     SELECT CustomerID FROM Bookings WHERE DurationDays > (
    ->         SELECT AVG(DurationDays) FROM Bookings
    ->     )
    -> );
+----------------+
| Name           |
+----------------+
| Priya Singh    |
| Rahul Verma    |
| Sneha Gupta    |
| Karan Malhotra |
| Rohan Joshi    |
+----------------+
5 rows in set (0.002 sec)
```

36. Find the destination details for the most expensive daily rate:

```
MariaDB [TravelBookingSystem]> SELECT * FROM Destinations WHERE PricePerDay = (
    ->     SELECT MAX(PricePerDay) FROM Destinations
    -> );
+---------------+----------+----------+-------------+
| DestinationID | CityName | Country  | PricePerDay |
+---------------+----------+----------+-------------+
|           110 | Malé     | Maldives |      350.00 |
+---------------+----------+----------+-------------+
1 row in set (0.001 sec)
```

37. Retrieve the name of the customer who made the highest single payment:

```
MariaDB [TravelBookingSystem]> SELECT Name FROM Customers WHERE CustomerID = (
    ->     SELECT CustomerID FROM Bookings WHERE BookingID = (
    ->         SELECT BookingID FROM Payments WHERE Amount = (
    ->             SELECT MAX(Amount) FROM Payments
    ->         )
    ->     )
    -> );
+-------------+
| Name        |
+-------------+
| Rahul Verma |
+-------------+
1 row in set (0.001 sec)
```

38. List destinations that currently have NO bookings (using subquery):

```
MariaDB [TravelBookingSystem]> SELECT CityName FROM Destinations WHERE DestinationID NOT IN (
    ->     SELECT DestinationID FROM Bookings
    -> );
Empty set (0.003 sec)
```

39. Find the second highest payment amount (using offset):

```
MariaDB [TravelBookingSystem]> SELECT Amount FROM Payments ORDER BY Amount DESC LIMIT 1 OFFSET 1;
+---------+
| Amount  |
+---------+
| 2100.00 |
+---------+
1 row in set (0.001 sec)
```

40. Calculate total cost of all trips to 'Japan' (using JOINs and aggregation):

```
MariaDB [TravelBookingSystem]> SELECT SUM(Payments.Amount) AS TotalRevenueFromJapan
    -> FROM Payments
    -> JOIN Bookings ON Payments.BookingID = Bookings.BookingID
    -> JOIN Destinations ON Bookings.DestinationID = Destinations.DestinationID
    -> WHERE Destinations.Country = 'Japan';
+-----------------------+
| TotalRevenueFromJapan |
+-----------------------+
|               1600.00 |
+-----------------------+
1 row in set (0.001 sec)
```

NORMALIZATION on Travel Booking System

Step 1: First Normal Form (1NF)

The Rule: All attributes must be atomic (indivisible), and there should be no repeating groups.

- **The Issue:** The Name column in the Customers table contains full names (e.g., "Amit Sharma"). If the agency wants to sort by last name or send personalized emails ("Hi Amit"), they have to use expensive string-splitting operations.
- **The Fix:** Split Name into FirstName and LastName.

Step 2: Second Normal Form (2NF)

The Rule: The database must be in 1NF, and all non-key attributes must be fully functionally dependent on the entire primary key (no partial dependencies).

- **The Assessment:** Because every table uses a single-column primary key (e.g., DestinationID instead of a composite key like CityName + Country), partial dependencies are mathematically impossible here.
- **The Fix:** None needed. The schema is safely in 2NF.

Step 3: Third Normal Form (3NF)

The Rule: The database must be in 2NF, and there must be no transitive dependencies (a non-primary key attribute depending on another non-primary key attribute) or derived data.

- **Issue 1 (Transitive Dependency):** In the Destinations table, Country depends on the CityName. If you update a city name or spell a country wrong, it creates anomalies.
    - *Fix:* Extract Country into its own Countries lookup table.
- **Issue 2 (Derived & Temporal Data):** In Payments, the Amount is technically derived from Bookings.DurationDays $\times$ Destinations.PricePerDay. However, if a destination raises its price in

2027, all your 2026 historical payment records will mathematically break if you rely on the Destinations table for calculation.

- *Fix:* We must store the LockedPricePerDay inside the Bookings table at the exact time the booking is made. This preserves historical financial integrity.

The New 3NF Normalized Schema

By applying these rules, our database expands to five tables to ensure absolute data integrity.

Table Name	Primary Key	Foreign Keys	Description
Countries	CountryID	None	New table to eliminate transitive geographical dependencies.
Customers	CustomerID	None	Updated with atomic FirstName and LastName columns.
Destinations	DestinationID	CountryID	Updated to link to Countries and tracks CurrentPricePerDay.
Bookings	BookingID	CustomerID, DestinationID	Updated to include LockedPricePerDay to protect historical pricing.
Payments	PaymentID	BookingID	Logs the actual AmountPaid against the booking.

```
MariaDB [TravelBookingSystem_Normalized]> CREATE TABLE Countries (
    ->     CountryID INT PRIMARY KEY,
    ->     CountryName VARCHAR(50) UNIQUE NOT NULL
    -> );
Query OK, 0 rows affected (0.028 sec)
```

```
MariaDB [travelbookingsystem]> CREATE DATABASE TravelBookingSystem_Normalized;
Query OK, 1 row affected (0.002 sec)

MariaDB [travelbookingsystem]> USE TravelBookingSystem_Normalized;
Database changed
```

```
MariaDB [TravelBookingSystem_Normalized]> CREATE TABLE Payments (
    ->     PaymentID INT PRIMARY KEY,
    ->     BookingID INT,
    ->     AmountPaid DECIMAL(10, 2),
    ->     PaymentMethod VARCHAR(50),
    ->     PaymentDate DATE,
    ->     FOREIGN KEY (BookingID) REFERENCES Bookings(BookingID)
    -> );
Query OK, 0 rows affected (0.040 sec)
```

```
MariaDB [TravelBookingSystem_Normalized]> CREATE TABLE Bookings (
    ->     BookingID INT PRIMARY KEY,
    ->     CustomerID INT,
    ->     DestinationID INT,
    ->     TravelDate DATE,
    ->     DurationDays INT,
    ->     LockedPricePerDay DECIMAL(10, 2) COMMENT 'Price at the time of booking',
    ->     FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID),
    ->     FOREIGN KEY (DestinationID) REFERENCES Destinations(DestinationID)
    -> );
Query OK, 0 rows affected (0.046 sec)
```

```
MariaDB [TravelBookingSystem_Normalized]> CREATE TABLE Destinations (
    ->     DestinationID INT PRIMARY KEY,
    ->     CityName VARCHAR(50) NOT NULL,
    ->     CountryID INT,
    ->     CurrentPricePerDay DECIMAL(10, 2),
    ->     FOREIGN KEY (CountryID) REFERENCES Countries(CountryID)
    -> );
Query OK, 0 rows affected (0.040 sec)
```

```
MariaDB [TravelBookingSystem_Normalized]> CREATE TABLE Customers (
    ->     CustomerID INT PRIMARY KEY,
    ->     FirstName VARCHAR(50) NOT NULL,
    ->     LastName VARCHAR(50) NOT NULL,
    ->     Email VARCHAR(100) UNIQUE,
    ->     Phone VARCHAR(15),
    ->     City VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.067 sec)
```